hw0

March 29, 2025

1 University of Chicago

1.0.1 DATA 25900: Ethics, Fairness, Responsibility, and Privacy in Data Science, Spring 2023

Course Staff: Satadisha Saha Bhowmick, Victor Brown, Jihee You

2 Assignment 0: Introduction

2.1 0. Resources to Start with Python

This is a list of resources with exercises that should help you refresh your programming skills in Python, the language we use in the class. Following and working through these exercises should feel comfortable, almost easy. If it is too hard, then you should consider getting up to speed ASAP so that your programming skills do not get in the way of complementing the programming assignments.

To start, this is a list of resources you should read

0. Get familiar with VSCode and Jupyter Notebook, so that you can download the assignment and work on it from your local Python environment. **We strongly recommend using VSCode since it will be a one-stop portal for you to develop your code and then export your notebook to a PDF for submission purposes**.

1. Python has its own official tutorial. Some other resources that might be easier for starters include: TutorialsPoint, W3Schools. For the tasks below, you probably need to know following things:

    - Basic Syntax of Python
    - Data Types (including list, tuple, dictionary)
    - If...else
    - Loops
    - Functions

2. A lot of interesting functionality (hypothesis testing, inferential stats, machine learning, and many more) are already available via libraries. Libraries are available in the Internet. You can install libraries in your environment to use them in your programs. You can use `pip` to install and manage all the packages you need. You can find the documentation here. If you are using Colab, many common libraries that are needed for the assignments are already pre-installed. If you need any other library, read this page on how to install libraries in Colab.

3. For data analysis, we will use a lot of a library called `pandas`. Its official documentation is here. The User Guide might be a good place to start.

4. For visualization, `pandas` has built-in visualization APIs, but you should become familiar with others. We recommend knowing `matplotlib` so you can do more advanced visualization. The documentation for matplotlib is here.

5. For statistical analysis, here is some potential resources if you have never taken a statistic class before:

    - A Gentle Introduction to Statistical Hypothesis Testing by Jason Brownlee

6. To do basic statistical analysis in Python, `scipy` is what most people would go to. You don't have to read the documentation thoroughly. Use it by searching based on needs should be enough.

2.2 0.1 Seeking Help

A key goal of this class is that you hone your programming skills. This not only means you become more proficient with Python and data analysis, but that you also become more resourceful to solve the problems you inevitably will find. Computer Science is a great community of researchers, practitioners, and aficionados who all together have been creating and publishing great content, mostly for free, for years. When you have a programming question that you do not know how to solve, you should leverage search engines and resources such as Stackoverflow to guide you to the solution. Of course, for these resources to be helpful, you need to ask the right question. Questions are good when they are concrete. For example, if I didn't know how to run a chi-square hypothesis test on some data, I could go to e.g., Google and write:

chi-square on two columns python

The first result points to stackoverflow, here, which contains both the original question (that matches ours) and a few answers.

As with any information coming from the outside world, you should scrutinize it carefully. You probably don't want to just copy-paste code. You want to make sure the question asked and yours match. If they do, then you want to ensure that the solution makes sense. Finally, you may want to consider the best way of incorporating the solution into your own program. As a general rule, if you don't fully understand what the solution does, then you should not use it. This is common sense, and yet, one of the main sources of pitfalls and problems in data analysis out there. Be responsible and own your work.

2.3 0.2 Deliverables

For each question, you may need to write Python code to do data analysis, provide only an explanation in prose, or do both. Use the *Code* block for any code you write in solving the problem, and use the *Markdown* block for your explanation of what you did. We have included one Code block for each question, but feel free to add more Code or Markdown blocks as you see fit. In general, within the Markdown block you should explain the approach you took in your code at a high level in addition to explicitly answering any questions that were asked. It is up to you to decide what code-based analyses, if any, are appropriate for a particular problem.

2.4 1. Load Data

To start with, please use pandas to load the file `confirmed_US.csv` as a dataframe. The is data about COVID cases in early 2020.

Some context on the data:

- **Lat:** Latitude
- **Long:** Longitude
- **Location:** The name of the location where the number of cases are reported.
- **1/22/2020, ..., 3/29/2020:** By the day indicated by the column name, how many cases have been reported in the location.

Note: if your dataframe is called `df`, you can simply display the dataframe by running the line `df` or `df.head(n)` with `n` being the number of rows you want to display.

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
DATA_DIR = '/Users/joycezhang/Downloads/Data259/PA0/'
```

[2]:
```
covid_df = pd.read_csv(DATA_DIR + 'confirmed_US.csv')
covid_df.head()
```

[2]:

	Lat	Long	Location	1/22/2020	1/23/2020	1/24/2020	\
0	NaN	NaN	Unassigned, Alabama, US	0	0	0	
1	NaN	NaN	Unassigned, Alaska, US	0	0	0	
2	NaN	NaN	Unassigned, Arizona, US	0	0	0	
3	NaN	NaN	Unassigned, Arkansas, US	0	0	0	
4	NaN	NaN	Unassigned, California, US	0	0	0	

	1/25/2020	1/26/2020	1/27/2020	1/28/2020	…	3/19/2020	3/20/2020	\
0	0	0	0	0	…	0	0	
1	0	0	0	0	…	0	0	
2	0	0	0	0	…	0	0	
3	0	0	0	0	…	58	96	
4	0	0	0	0	…	0	0	

	3/21/2020	3/23/2020	3/24/2020	3/25/2020	3/26/2020	3/27/2020	\
0	0	0	0	0	0	0	
1	0	0	0	0	0	0	
2	0	0	0	0	0	0	
3	118	3	13	20	24	35	
4	0	0	0	0	0	12	

	3/28/2020	3/29/2020
0	0	0
1	0	0
2	0	0

```
3          32          31
4           0           0

[5 rows x 70 columns]
```

[3]:
```
covid_df['Location'] = covid_df['Location'].apply(str)
```

2.5 2. Basic Analysis

1. The location of reported cases are in a combined string right now. To make analysis easier, please create a column for counties and another column for states.

   Please be aware that there will be special cases in the dataset, such as Diamond Princess, etc. The location that doesn't have a county will be regarded as a state and a `None` should be placed as its county.

[4]:
```
def get_county_state(location_str):
    # you may find it helpful to create a function that extracts county and␣
 ↪state (and deal with special cases) from a location string
    separated_str = [item.strip() for item in location_str.split(',')]
    if len(separated_str) > 2:
        county = separated_str[0]
        state =  separated_str[1]
    else:
        county = None
        state = separated_str[0]
    return [county, state]
```

[5]:
```
covid_df[['County', 'State']] = covid_df['Location'].apply(get_county_state).
 ↪apply(pd.Series)
```

[6]:
```
covid_df.head()
```

[6]:
```
   Lat  Long                     Location  1/22/2020  1/23/2020  1/24/2020  \
0  NaN   NaN     Unassigned, Alabama, US          0          0          0
1  NaN   NaN      Unassigned, Alaska, US          0          0          0
2  NaN   NaN     Unassigned, Arizona, US          0          0          0
3  NaN   NaN    Unassigned, Arkansas, US          0          0          0
4  NaN   NaN  Unassigned, California, US          0          0          0

   1/25/2020  1/26/2020  1/27/2020  1/28/2020  …  3/21/2020  3/23/2020  \
0          0          0          0          0  …          0          0
1          0          0          0          0  …          0          0
2          0          0          0          0  …          0          0
3          0          0          0          0  …        118          3
4          0          0          0          0  …          0          0

   3/24/2020  3/25/2020  3/26/2020  3/27/2020  3/28/2020  3/29/2020  \
```

```
0          0          0          0          0          0          0
1          0          0          0          0          0          0
2          0          0          0          0          0          0
3         13         20         24         35         32         31
4          0          0          0         12          0          0

       County       State
0  Unassigned     Alabama
1  Unassigned      Alaska
2  Unassigned     Arizona
3  Unassigned    Arkansas
4  Unassigned  California

[5 rows x 72 columns]
```

2. Please select all the rows where the state is Illinois and sort the rows based on the number of confirmed cases on 3/29/2020 from the most to the least.

[7]:
```
IL_df = covid_df[covid_df['State']=='Illinois'].copy().sort_values(by='3/29/
↪2020', ascending=False)
IL_df = IL_df.drop(IL_df.columns[:3], axis=1)
IL_df
```

[7]:
```
     1/22/2020  1/23/2020  1/24/2020  1/25/2020  1/26/2020  1/27/2020  \
661          0          0          1          1          1          1
694          0          0          0          0          0          0
667          0          0          0          0          0          0
744          0          0          0          0          0          0
690          0          0          0          0          0          0
..         …          …          …          …          …          …
706          0          0          0          0          0          0
669          0          0          0          0          0          0
709          0          0          0          0          0          0
711          0          0          0          0          0          0
696          0          0          0          0          0          0

     1/28/2020  1/29/2020  1/30/2020  1/31/2020  …  3/21/2020  3/23/2020  \
661          1          1          1          2  …        548        922
694          0          0          0          0  …         63         96
667          0          0          0          0  …         65         95
744          0          0          0          0  …         12         24
690          0          0          0          0  …          8         18
..         …          …          …          …  …          …          …
706          0          0          0          0  …          0          0
669          0          0          0          0  …          0          0
709          0          0          0          0  …          0          0
711          0          0          0          0  …          0          0
696          0          0          0          0  …          0          0
```

```
     3/24/2020  3/25/2020  3/26/2020  3/27/2020  3/28/2020  3/29/2020  \
661       1194       1418       1418       2239       2613       3445
694        111        139        139        230        241        300
667        103        131        131        199        202        274
744         28         40         40        104        127        182
690         24         38         38         75         90        100
..         …          …          …          …          …          …
706          0          0          0          0          0          0
669          0          0          0          0          0          0
709          0          0          0          0          0          0
711          0          0          0          0          0          0
696          0          0          0          0          0          0

       County     State
661      Cook  Illinois
694      Lake  Illinois
667    DuPage  Illinois
744      Will  Illinois
690      Kane  Illinois
..        …       …
706    Marion  Illinois
669   Edwards  Illinois
709    Massac  Illinois
711    Mercer  Illinois
696  Lawrence  Illinois

[103 rows x 69 columns]
```

3. Which county in Illinois is the 10th county with the most cases by 3/29/2020? How many cases were there on 3/29/2020?

[8]:
```
IL_df.set_index('County', inplace=True)
cols = IL_df.columns.values[:-1]
IL_df = IL_df[cols].cumsum(axis=1)
IL_df.head(10)
```

[8]:
```
            1/22/2020  1/23/2020  1/24/2020  1/25/2020  1/26/2020  1/27/2020  \
County
Cook                0          0          1          2          3          4
Lake                0          0          0          0          0          0
DuPage              0          0          0          0          0          0
Will                0          0          0          0          0          0
Kane                0          0          0          0          0          0
McHenry             0          0          0          0          0          0
Kankakee            0          0          0          0          0          0
St. Clair           0          0          0          0          0          0
Champaign           0          0          0          0          0          0
```

```
Kendall            0          0          0          0          0          0

           1/28/2020  1/29/2020  1/30/2020  1/31/2020  …  3/19/2020  \
County                                                 …
Cook               5          6          7          9  …        937
Lake               0          0          0          0  …         76
DuPage             0          0          0          0  …        128
Will               0          0          0          0  …         15
Kane               0          0          0          0  …         29
McHenry            0          0          0          0  …         23
Kankakee           0          0          0          0  …          1
St. Clair          0          0          0          0  …         16
Champaign          0          0          0          0  …          4
Kendall            0          0          0          0  …          3

           3/20/2020  3/21/2020  3/23/2020  3/24/2020  3/25/2020  3/26/2020  \
County
Cook            1215       1763       2685       3879       5297       6715
Lake             113        176        272        383        522        661
DuPage           182        247        342        445        576        707
Will              24         36         60         88        128        168
Kane              35         43         61         85        123        161
McHenry           29         40         52         66         85        104
Kankakee           2          4          7         11         17         23
St. Clair         19         22         26         33         40         47
Champaign          5          8         11         15         19         29
Kendall            4          6         10         14         20         26

           3/27/2020  3/28/2020  3/29/2020
County
Cook            8954      11567      15012
Lake             891       1132       1432
DuPage           906       1108       1382
Will             272        399        581
Kane             236        326        426
McHenry          149        196        248
Kankakee          40         67        100
St. Clair         62         80        111
Champaign         39         50         71
Kendall           34         45         60

[10 rows x 67 columns]
```

Kendall county had the 10th most cases by March 29th 2020. There were 15 cases on March 29th 2020.

4. Please load data from `states_population_2019.csv` and assume the population doesn't change since then. Then, the population data should be joined to the dataframe you got in

Question 1 based on `State` as a new column called `State Population`. If you have never taken a Database class before, the link here will help you understand what "join" means.

For rows where the data in `State` is not actually a state (e.g. Diamond Princess), feel free to just leave it with a `NaN`.

[9]:
```
pop_df = pd.read_csv(DATA_DIR + 'states_population_2019.csv')
joined_df = pd.merge(covid_df, pop_df, on='State', how='left')
print(joined_df.shape) #ensure we are merging correctly :D
joined_df.head()
```

```
(3205, 73)
```

[9]:

	Lat	Long	Location	1/22/2020	1/23/2020	1/24/2020	\
0	NaN	NaN	Unassigned, Alabama, US	0	0	0	
1	NaN	NaN	Unassigned, Alaska, US	0	0	0	
2	NaN	NaN	Unassigned, Arizona, US	0	0	0	
3	NaN	NaN	Unassigned, Arkansas, US	0	0	0	
4	NaN	NaN	Unassigned, California, US	0	0	0	

	1/25/2020	1/26/2020	1/27/2020	1/28/2020	…	3/23/2020	3/24/2020	\
0	0	0	0	0	…	0	0	
1	0	0	0	0	…	0	0	
2	0	0	0	0	…	0	0	
3	0	0	0	0	…	3	13	
4	0	0	0	0	…	0	0	

	3/25/2020	3/26/2020	3/27/2020	3/28/2020	3/29/2020	County	\
0	0	0	0	0	0	Unassigned	
1	0	0	0	0	0	Unassigned	
2	0	0	0	0	0	Unassigned	
3	20	24	35	32	31	Unassigned	
4	0	0	12	0	0	Unassigned	

	State	Population
0	Alabama	4903185.0
1	Alaska	731545.0
2	Arizona	7278717.0
3	Arkansas	3017825.0
4	California	39512223.0

[5 rows x 73 columns]

5. What are the top 5 states in the US that have the most confirmed cases by 3/29/2020?

[10]:
```
joined_df['cum_sum'] = joined_df.iloc[:, 3:-3].sum(axis=1)
joined_df.head()
```

[10]:
```
   Lat  Long                    Location  1/22/2020  1/23/2020  1/24/2020  \
0  NaN   NaN     Unassigned, Alabama, US          0          0          0
1  NaN   NaN      Unassigned, Alaska, US          0          0          0
2  NaN   NaN     Unassigned, Arizona, US          0          0          0
3  NaN   NaN    Unassigned, Arkansas, US          0          0          0
4  NaN   NaN  Unassigned, California, US          0          0          0

   1/25/2020  1/26/2020  1/27/2020  1/28/2020  …  3/24/2020  3/25/2020  \
0          0          0          0          0  …          0          0
1          0          0          0          0  …          0          0
2          0          0          0          0  …          0          0
3          0          0          0          0  …         13         20
4          0          0          0          0  …          0          0

   3/26/2020  3/27/2020  3/28/2020  3/29/2020      County       State  \
0          0          0          0          0  Unassigned     Alabama
1          0          0          0          0  Unassigned      Alaska
2          0          0          0          0  Unassigned     Arizona
3         24         35         32         31  Unassigned    Arkansas
4          0         12          0          0  Unassigned  California

   Population  cum_sum
0   4903185.0        4
1    731545.0        1
2   7278717.0        0
3   3017825.0      448
4  39512223.0       13

[5 rows x 74 columns]
```

[11]:
```
state_df = joined_df.groupby('State')['cum_sum'].sum().
 ↪sort_values(ascending=False)
state_df.head()
```

[11]:
```
State
New York      287866
New Jersey     55188
California     35173
Washington     33104
Michigan       23549
Name: cum_sum, dtype: int64
```

Top 5 states are New York, New Jersey, Washington, California, and Illinois.

6. For each state in the US, display the county that has the most confirmed cases by 3/29/2020?

[12]:
```
top_county = joined_df.groupby(['State', 'County'])['cum_sum'].sum().
 ↪reset_index()
```

```
top_county.loc[top_county.groupby('State')['cum_sum'].idxmax()]
```

[12]:

	State	County	cum_sum
36	Alabama	Jefferson	1297
70	Alaska	Anchorage	220
105	Arizona	Maricopa	2486
173	Arkansas	Pulaski	559
208	California	Los Angeles	9577
265	Colorado	Denver	2151
314	Connecticut	Fairfield	5249
324	Delaware	New Castle	796
327	District of Columbia	District of Columbia	1819
371	Florida	Miami-Dade	4912
456	Georgia	Fulton	2265
558	Hawaii	Honolulu	620
569	Idaho	Blaine	549
623	Illinois	Cook	15012
759	Indiana	Marion	2642
855	Iowa	Johnson	547
949	Kansas	Johnson	541
1122	Kentucky	Unassigned	640
1166	Louisiana	Orleans	8425
1198	Maine	Cumberland	828
1228	Maryland	Montgomery	1526
1247	Massachusetts	Middlesex	5051
1337	Michigan	Wayne	11137
1365	Minnesota	Hennepin	1112
1443	Mississippi	DeSoto	357
1611	Missouri	St. Louis	1423
1642	Montana	Gallatin	251
1711	Nebraska	Douglas	501
1780	Nevada	Clark	2861
1803	New Hampshire	Rockingham	513
1826	New Jersey	Unassigned	11294
1829	New Mexico	Bernalillo	565
1893	New York	New York City	158259
1985	North Carolina	Mecklenburg	1698
2034	North Dakota	Burleigh	196
2098	Ohio	Cuyahoga	2265
2224	Oklahoma	Oklahoma	591
2282	Oregon	Washington	1009
2335	Pennsylvania	Philadelphia	3847
2356	Rhode Island	Providence	854
2386	South Carolina	Kershaw	684
2454	South Dakota	Minnehaha	147
2491	Tennessee	Davidson	2539
2626	Texas	Dallas	2515

2843	Utah	Salt Lake	1581
2859	Vermont	Chittenden	565
2908	Virginia	Fairfax	843
3021	Washington	King	17271
3075	West Virginia	Monongalia	119
3141	Wisconsin	Milwaukee	3058
3180	Wyoming	Fremont	152

Starting from here, we will ask you to make some visualizations (plots). For **all** the figures below, please include a title of the figure, and name both the x-axis and the y-axis. All the labels on the x-axis and the y-axis should be readable (i.e. no overlaps).

---

7. Please create a line plot to show how the cumulative number of confirmed cases changed in the US through time.

[13]:
```
cum_sum = covid_df.iloc[:, 3:-2].sum().cumsum().tolist()
cum_sum
```

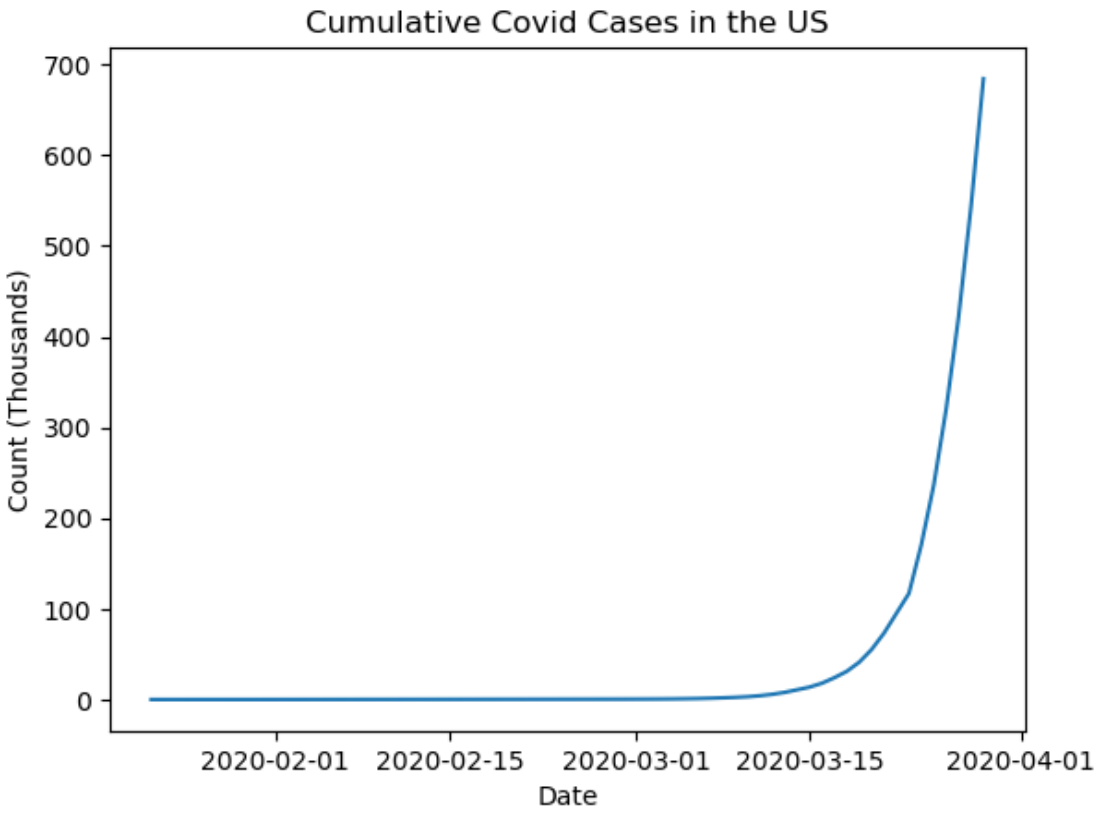
[13]:
```
[1,
 2,
 4,
 6,
 11,
 16,
 21,
 26,
 31,
 38,
 46,
 54,
 65,
 76,
 87,
 98,
 109,
 120,
 131,
 142,
 154,
 166,
 179,
 192,
 205,
 218,
 231,
```

```
244,
257,
270,
285,
300,
315,
330,
345,
360,
376,
392,
416,
446,
499,
572,
676,
848,
1065,
1401,
1851,
2365,
3073,
4178,
5735,
7882,
10739,
13657,
17964,
24060,
31062,
41132,
55211,
73307,
116971,
170709,
236490,
320329,
421990,
543476,
684370]
```

[14]:
```
y = [x/1000 for x in cum_sum]
date_columns = covid_df.columns.values[3:-2].copy()
date_columns = pd.to_datetime(covid_df.columns.values[3:-2])
plt.plot(date_columns, y)
plt.title('Cumulative Covid Cases in the US')
plt.xlabel('Date')
```

```
plt.ylabel('Count (Thousands)')
```

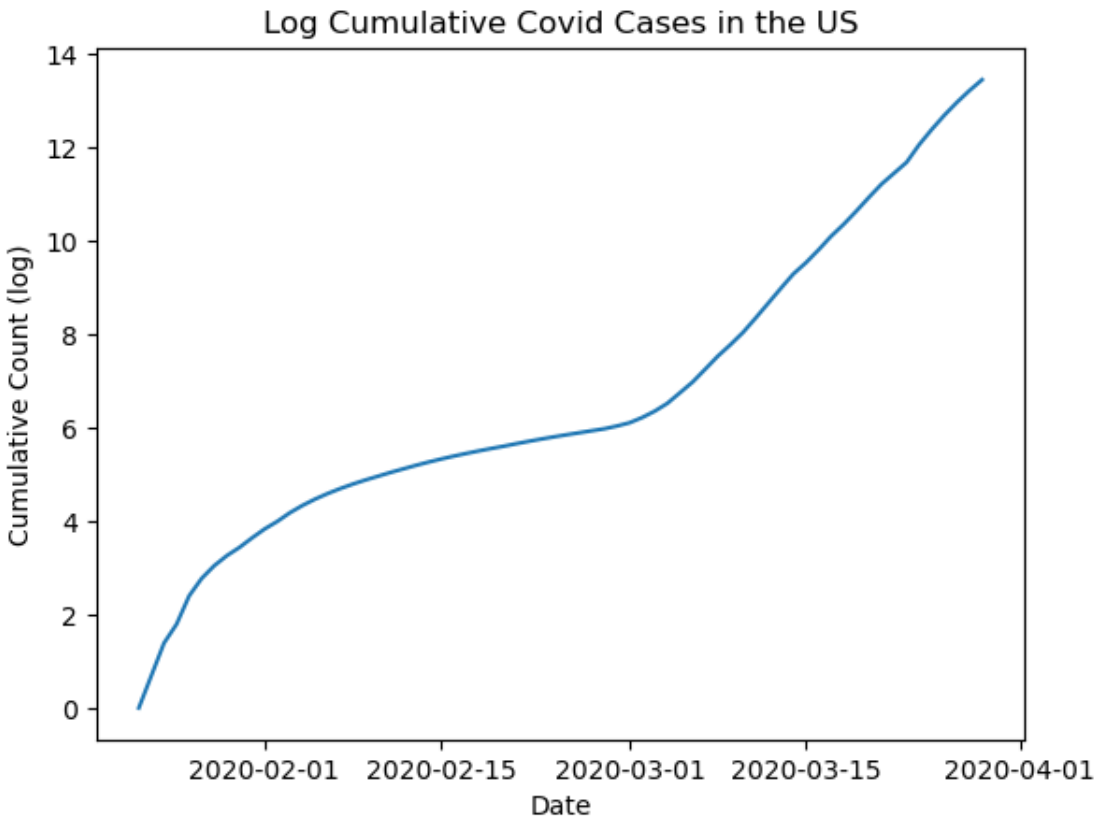
[14]: `Text(0, 0.5, 'Count (Thousands)')`



8. What if we put the y-axis in the above figure to log?

[15]:
```
cum_sum_log = [np.log(x) for x in cum_sum]
plt.plot(date_columns, cum_sum_log)
plt.title('Log Cumulative Covid Cases in the US')
plt.xlabel('Date')
plt.ylabel('Cumulative Count (log)')
```

[15]: `Text(0, 0.5, 'Cumulative Count (log)')`



9. Please create a bar plot to display the states in the US that have the ten highest densities of confirmed cases by 3/29/2020.

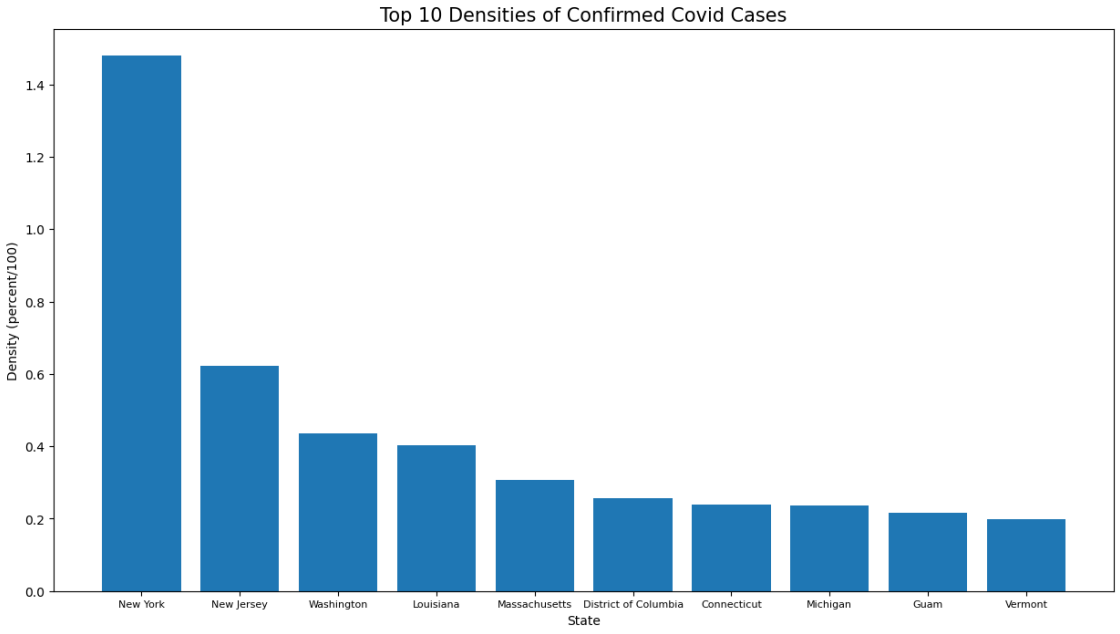
[16]:
```
density_df = pd.merge(state_df, pop_df, on='State', how='inner')
density_df['Density'] = density_df['cum_sum']/density_df['Population']*100
top_10_density = density_df[['Density', 'State']].sort_values(by='Density',␣
↪ascending=False).head(10)
top_10_density
```

[16]:

	Density	State
0	1.479760	New York
1	0.621333	New Jersey
3	0.434727	Washington
8	0.403008	Louisiana
5	0.306828	Massachusetts
33	0.257740	District of Columbia
14	0.239728	Connecticut
4	0.235800	Michigan
52	0.215426	Guam
40	0.198401	Vermont

[17]:
```
#bar plot
plt.figure(figsize=(15,8))
plt.bar(x='State', height='Density', data=top_10_density)
plt.title('Top 10 Densities of Confirmed Covid Cases', fontsize=15)
plt.xlabel('State')
plt.xticks(fontsize=8)
plt.ylabel('Density (percent/100)')
```

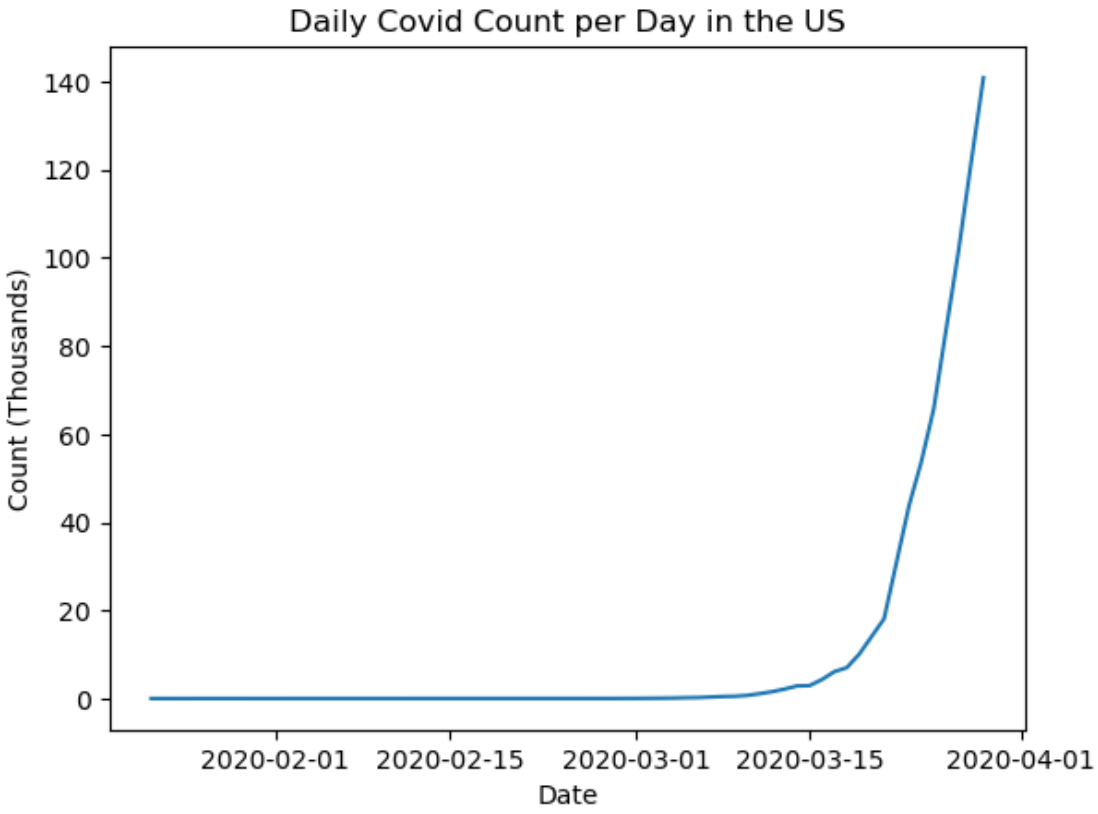
[17]:
```
Text(0, 0.5, 'Density (percent/100)')
```



10. Please create a line plot to show the increase of confirmed cases for each day in the US.

[18]:
```
daily_counts = [cum_sum[0]] + [cum_sum[i] - cum_sum[i-1] for i in range(1,␣
 ↪len(cum_sum))]
y = [x/1000 for x in daily_counts]
plt.plot(date_columns, y)
plt.title('Daily Covid Count per Day in the US')
plt.xlabel('Date')
plt.ylabel('Count (Thousands)')
```

[18]:
```
Text(0, 0.5, 'Count (Thousands)')
```



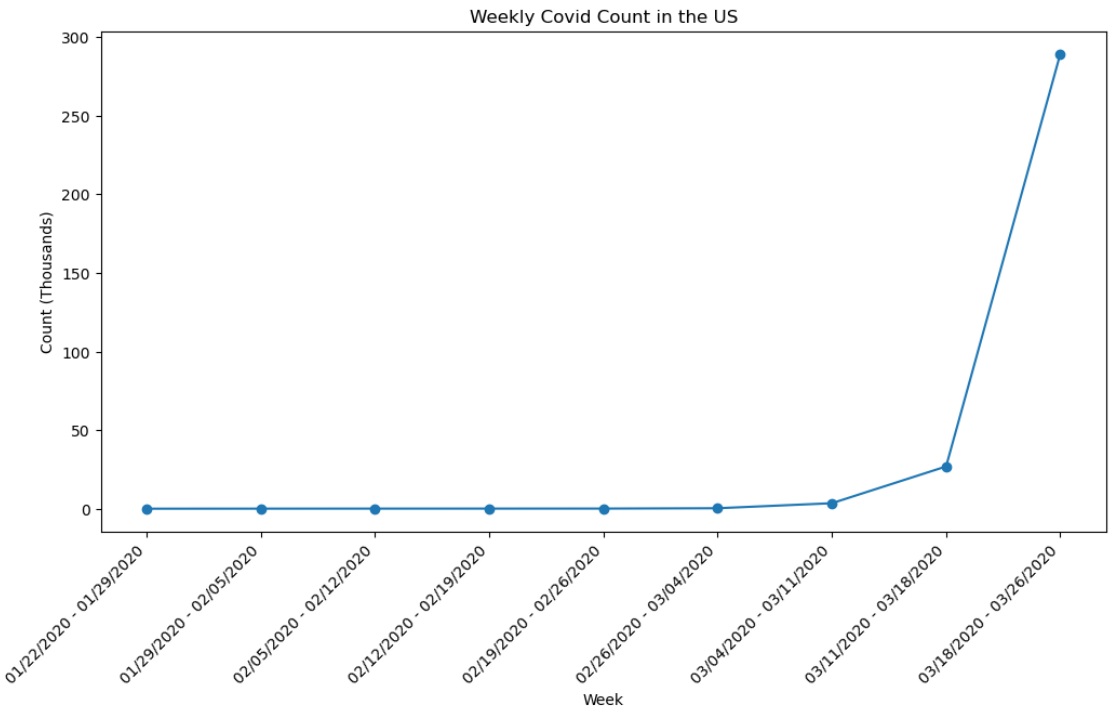
11. Please create a line plot to show the increase of confirmed cases for each week in the US. You can use the increase between 1/22 and 1/29 as the starting point and 1/29-2/5 as the next.

    Please make sure that the date on the x-axis is readable and all of them are fomated such as `01/22/2020 - 01/29/2020`. (Hint: you can rotate the labels on x-axis)

[19]:
```
weekly_cum_sum = cum_sum[::7]
weekly_increase = [weekly_cum_sum[i] - weekly_cum_sum[i-1] for i in range(1,␣
 ↪len(weekly_cum_sum))]

weekly_dates = pd.to_datetime(covid_df.columns.values[3:-2][::7]).strftime('%m/
 ↪%d/%Y')
x_labels = [f"{weekly_dates[i]} - {weekly_dates[i+1]}" for i in␣
 ↪range(len(weekly_dates) - 1)]

y = [x/1000 for x in weekly_increase]

plt.figure(figsize=(12, 6))
plt.plot(x_labels, y, marker='o', linestyle='-')
plt.title('Weekly Covid Count in the US')
plt.xlabel('Week')
```

```
plt.xticks(rotation=45, ha="right")
plt.ylabel('Count (Thousands)')
```

[19]:
```
Text(0, 0.5, 'Count (Thousands)')
```



12. Please create a line plot that shows the changes of confirmed cases through time in top 10 states with the most confirmed cases by 3/29/2020. The y-axis of the plot should be in log. The 10 lines should be in different colors, with a legend specifying which color corresponds to which state.

[20]:
```
grouped = covid_df.groupby('State')
date_cols = covid_df.columns[3:-2]
state_sum = grouped[date_cols].sum()
state_sum
```

[20]:

State	1/22/2020	1/23/2020	1/24/2020	1/25/2020	\
Alabama	0	0	0	0	
Alaska	0	0	0	0	
American Samoa	0	0	0	0	
Arizona	0	0	0	0	
Arkansas	0	0	0	0	
California	0	0	0	0	
Colorado	0	0	0	0	

Connecticut	0	0	0	0
Delaware	0	0	0	0
Diamond Princess	0	0	0	0
District of Columbia	0	0	0	0
Florida	0	0	0	0
Georgia	0	0	0	0
Grand Princess	0	0	0	0
Guam	0	0	0	0
Hawaii	0	0	0	0
Idaho	0	0	0	0
Illinois	0	0	1	1
Indiana	0	0	0	0
Iowa	0	0	0	0
Kansas	0	0	0	0
Kentucky	0	0	0	0
Louisiana	0	0	0	0
Maine	0	0	0	0
Maryland	0	0	0	0
Massachusetts	0	0	0	0
Michigan	0	0	0	0
Minnesota	0	0	0	0
Mississippi	0	0	0	0
Missouri	0	0	0	0
Montana	0	0	0	0
Nebraska	0	0	0	0
Nevada	0	0	0	0
New Hampshire	0	0	0	0
New Jersey	0	0	0	0
New Mexico	0	0	0	0
New York	0	0	0	0
North Carolina	0	0	0	0
North Dakota	0	0	0	0
Northern Mariana Islands	0	0	0	0
Ohio	0	0	0	0
Oklahoma	0	0	0	0
Oregon	0	0	0	0
Pennsylvania	0	0	0	0
Puerto Rico	0	0	0	0
Rhode Island	0	0	0	0
South Carolina	0	0	0	0
South Dakota	0	0	0	0
Tennessee	0	0	0	0
Texas	0	0	0	0
Utah	0	0	0	0
Vermont	0	0	0	0
Virgin Islands	0	0	0	0
Virginia	0	0	0	0

Washington	1	1	1	1	
West Virginia	0	0	0	0	
Wisconsin	0	0	0	0	
Wyoming	0	0	0	0	

State	1/26/2020	1/27/2020	1/28/2020	1/29/2020	\
Alabama	0	0	0	0	
Alaska	0	0	0	0	
American Samoa	0	0	0	0	
Arizona	1	1	1	1	
Arkansas	0	0	0	0	
California	2	2	2	2	
Colorado	0	0	0	0	
Connecticut	0	0	0	0	
Delaware	0	0	0	0	
Diamond Princess	0	0	0	0	
District of Columbia	0	0	0	0	
Florida	0	0	0	0	
Georgia	0	0	0	0	
Grand Princess	0	0	0	0	
Guam	0	0	0	0	
Hawaii	0	0	0	0	
Idaho	0	0	0	0	
Illinois	1	1	1	1	
Indiana	0	0	0	0	
Iowa	0	0	0	0	
Kansas	0	0	0	0	
Kentucky	0	0	0	0	
Louisiana	0	0	0	0	
Maine	0	0	0	0	
Maryland	0	0	0	0	
Massachusetts	0	0	0	0	
Michigan	0	0	0	0	
Minnesota	0	0	0	0	
Mississippi	0	0	0	0	
Missouri	0	0	0	0	
Montana	0	0	0	0	
Nebraska	0	0	0	0	
Nevada	0	0	0	0	
New Hampshire	0	0	0	0	
New Jersey	0	0	0	0	
New Mexico	0	0	0	0	
New York	0	0	0	0	
North Carolina	0	0	0	0	
North Dakota	0	0	0	0	
Northern Mariana Islands	0	0	0	0	

Ohio	0	0	0	0	
Oklahoma	0	0	0	0	
Oregon	0	0	0	0	
Pennsylvania	0	0	0	0	
Puerto Rico	0	0	0	0	
Rhode Island	0	0	0	0	
South Carolina	0	0	0	0	
South Dakota	0	0	0	0	
Tennessee	0	0	0	0	
Texas	0	0	0	0	
Utah	0	0	0	0	
Vermont	0	0	0	0	
Virgin Islands	0	0	0	0	
Virginia	0	0	0	0	
Washington	1	1	1	1	
West Virginia	0	0	0	0	
Wisconsin	0	0	0	0	
Wyoming	0	0	0	0	

	1/30/2020	1/31/2020	…	3/19/2020	3/20/2020	\
State			…			
Alabama	0	0	…	78	106	
Alaska	0	0	…	8	11	
American Samoa	0	0	…	0	0	
Arizona	1	1	…	45	68	
Arkansas	0	0	…	62	100	
California	2	3	…	1005	1243	
Colorado	0	0	…	277	363	
Connecticut	0	0	…	159	194	
Delaware	0	0	…	30	39	
Diamond Princess	0	0	…	47	49	
District of Columbia	0	0	…	71	77	
Florida	0	0	…	331	440	
Georgia	0	0	…	287	485	
Grand Princess	0	0	…	22	23	
Guam	0	0	…	12	14	
Hawaii	0	0	…	25	36	
Idaho	0	0	…	23	36	
Illinois	1	2	…	422	585	
Indiana	0	0	…	60	86	
Iowa	0	0	…	44	45	
Kansas	0	0	…	35	48	
Kentucky	0	0	…	50	64	
Louisiana	0	0	…	392	537	
Maine	0	0	…	52	56	
Maryland	0	0	…	107	149	
Massachusetts	0	0	…	328	413	

Michigan	0	0	…	259	402
Minnesota	0	0	…	89	113
Mississippi	0	0	…	50	80
Missouri	0	0	…	39	73
Montana	0	0	…	11	20
Nebraska	0	0	…	29	37
Nevada	0	0	…	95	161
New Hampshire	0	0	…	42	53
New Jersey	0	0	…	741	890
New Mexico	0	0	…	35	42
New York	0	0	…	1750	3252
North Carolina	0	0	…	136	189
North Dakota	0	0	…	19	26
Northern Mariana Islands	0	0	…	0	0
Ohio	0	0	…	119	173
Oklahoma	0	0	…	44	49
Oregon	0	0	…	88	114
Pennsylvania	0	0	…	206	311
Puerto Rico	0	0	…	5	14
Rhode Island	0	0	…	44	54
South Carolina	0	0	…	80	126
South Dakota	0	0	…	14	14
Tennessee	0	0	…	154	223
Texas	0	0	…	306	374
Utah	0	0	…	62	95
Vermont	0	0	…	22	29
Virgin Islands	0	0	…	3	3
Virginia	0	0	…	103	123
Washington	1	1	…	1374	1524
West Virginia	0	0	…	2	8
Wisconsin	0	0	…	159	219
Wyoming	0	0	…	18	21

	3/21/2020	3/23/2020	3/24/2020	3/25/2020	\
State					
Alabama	131	196	242	381	
Alaska	13	30	34	41	
American Samoa	0	0	0	0	
Arizona	104	235	326	401	
Arkansas	122	192	219	280	
California	1405	2108	2538	2998	
Colorado	475	704	723	1021	
Connecticut	194	415	618	875	
Delaware	45	68	104	119	
Diamond Princess	49	49	49	49	
District of Columbia	0	120	141	187	
Florida	763	1227	1412	1682	

Georgia	555	772	1026	1247
Grand Princess	23	28	28	28
Guam	15	29	32	37
Hawaii	36	56	90	91
Idaho	42	68	81	91
Illinois	753	1285	1537	1865
Indiana	128	270	368	477
Iowa	69	105	124	146
Kansas	57	84	100	134
Kentucky	87	123	162	197
Louisiana	763	1172	1388	1795
Maine	70	107	118	142
Maryland	193	290	349	425
Massachusetts	522	778	1161	1841
Michigan	540	1329	1793	2296
Minnesota	134	234	261	286
Mississippi	140	249	320	377
Missouri	81	187	257	354
Montana	27	34	51	65
Nebraska	38	51	66	71
Nevada	161	245	278	323
New Hampshire	61	101	101	108
New Jersey	1327	2844	3675	4402
New Mexico	55	83	100	113
New York	4197	20884	25681	30841
North Carolina	256	353	495	590
North Dakota	28	30	36	45
Northern Mariana Islands	0	0	0	0
Ohio	247	443	567	704
Oklahoma	53	81	106	164
Oregon	137	191	210	266
Pennsylvania	399	698	946	1260
Puerto Rico	21	31	39	51
Rhode Island	66	106	124	132
South Carolina	174	298	342	424
South Dakota	14	28	30	41
Tennessee	304	614	772	916
Texas	582	758	955	1229
Utah	118	257	298	340
Vermont	49	75	95	125
Virgin Islands	6	7	17	17
Virginia	157	254	293	396
Washington	1793	2221	2328	2591
West Virginia	12	16	22	39
Wisconsin	282	425	481	621
Wyoming	23	26	29	44

State	3/26/2020	3/27/2020	3/28/2020	3/29/2020
Alabama	517	587	694	825
Alaska	56	58	85	102
American Samoa	0	0	0	0
Arizona	508	665	773	919
Arkansas	335	381	409	426
California	3899	4657	5095	5852
Colorado	1430	1433	1740	2307
Connecticut	1012	1291	1524	1993
Delaware	130	163	214	232
Diamond Princess	49	49	49	49
District of Columbia	231	271	304	342
Florida	2357	2900	3763	4246
Georgia	1525	2000	2366	2651
Grand Princess	28	28	103	103
Guam	45	51	55	56
Hawaii	95	106	149	149
Idaho	146	205	234	281
Illinois	2538	3024	3491	4596
Indiana	645	979	1233	1513
Iowa	179	235	298	336
Kansas	172	206	266	330
Kentucky	247	301	393	438
Louisiana	2304	2744	3315	3540
Maine	155	168	211	253
Maryland	583	775	995	1239
Massachusetts	2420	3244	4265	4963
Michigan	2845	3634	4650	5488
Minnesota	344	396	441	503
Mississippi	485	579	663	759
Missouri	520	666	836	915
Montana	90	109	129	154
Nebraska	74	82	96	108
Nevada	420	536	626	920
New Hampshire	137	158	187	214
New Jersey	6876	8825	11124	13386
New Mexico	113	136	208	237
New York	37877	44876	52410	59648
North Carolina	738	887	1020	1191
North Dakota	51	68	94	98
Northern Mariana Islands	0	0	0	0
Ohio	868	1137	1406	1653
Oklahoma	248	322	377	429
Oregon	316	416	479	548
Pennsylvania	1795	2345	2845	3432
Puerto Rico	64	79	100	127

```
Rhode Island                  165         203         239         294
South Carolina                424         542         660         774
South Dakota                   46          58          68          90
Tennessee                    1097        1318        1511        1720
Texas                        1563        1937        2455        2792
Utah                          396         472         602         720
Vermont                       158         184         211         235
Virgin Islands                 17          19          22           0
Virginia                      466         607         740         890
Washington                   3207        3477        4030        4465
West Virginia                  52          76          96         113
Wisconsin                     728         926        1055        1164
Wyoming                        53          70          82          86

[58 rows x 67 columns]
```

[21]:
```
state_df.head(10) # now we know our top states
```

[21]:
```
State
New York          287866
New Jersey         55188
California         35173
Washington         33104
Michigan           23549
Massachusetts      21323
Illinois           20969
Florida            20054
Louisiana          18735
Pennsylvania       14785
Name: cum_sum, dtype: int64
```

[22]:
```
top_state = [
    'New York',
    'New Jersey',
    'California',
    'Washington',
    'Michigan',
    'Massachusetts',
    'Illinois',
    'Florida',
    'Louisiana',
    'Pennsylvania'
] # this is taken from my states_df above

state_10 = state_sum[state_sum.index.isin(top_state)].sort_values(by='State')␣
 ↪#alphabetical order
cum_sum_state = state_10.cumsum(axis=1)
```

```
cum_sum_state
```

[22]:

State	1/22/2020	1/23/2020	1/24/2020	1/25/2020	1/26/2020	\
California	0	0	0	0	2	
Florida	0	0	0	0	0	
Illinois	0	0	1	2	3	
Louisiana	0	0	0	0	0	
Massachusetts	0	0	0	0	0	
Michigan	0	0	0	0	0	
New Jersey	0	0	0	0	0	
New York	0	0	0	0	0	
Pennsylvania	0	0	0	0	0	
Washington	1	2	3	4	5	

State	1/27/2020	1/28/2020	1/29/2020	1/30/2020	1/31/2020	…	\
California	4	6	8	10	13	…	
Florida	0	0	0	0	0	…	
Illinois	4	5	6	7	9	…	
Louisiana	0	0	0	0	0	…	
Massachusetts	0	0	0	0	0	…	
Michigan	0	0	0	0	0	…	
New Jersey	0	0	0	0	0	…	
New York	0	0	0	0	0	…	
Pennsylvania	0	0	0	0	0	…	
Washington	6	7	8	9	10	…	

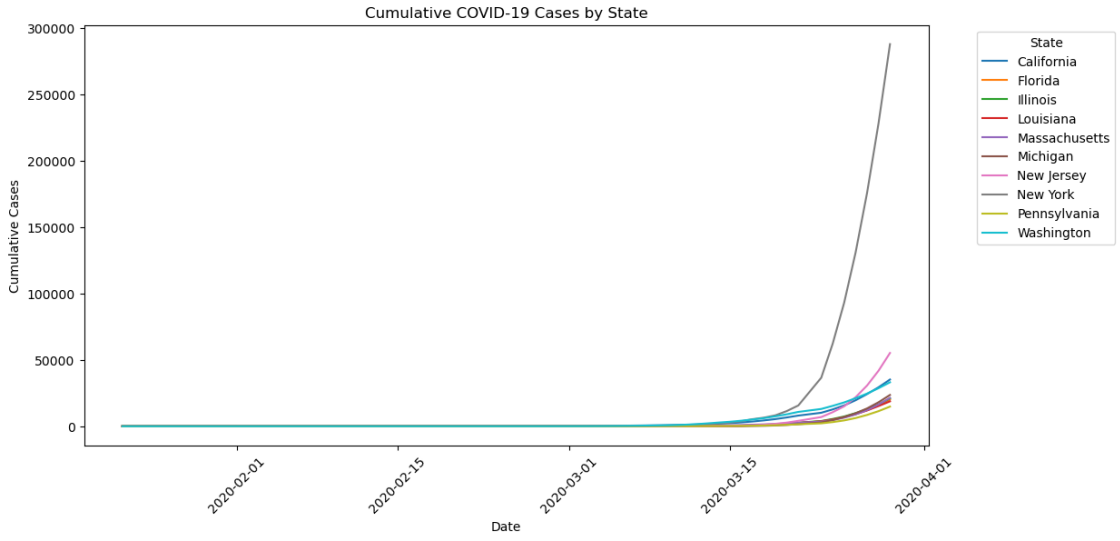
State	3/19/2020	3/20/2020	3/21/2020	3/23/2020	3/24/2020	\
California	5378	6621	8026	10134	12672	
Florida	1264	1704	2467	3694	5106	
Illinois	1295	1880	2633	3918	5455	
Louisiana	1177	1714	2477	3649	5037	
Massachusetts	1716	2129	2651	3429	4590	
Michigan	572	974	1514	2843	4636	
New Jersey	1839	2729	4056	6900	10575	
New York	8200	11452	15649	36533	62214	
Pennsylvania	754	1065	1464	2162	3108	
Washington	7468	8992	10785	13006	15334	

State	3/25/2020	3/26/2020	3/27/2020	3/28/2020	3/29/2020
California	15670	19569	24226	29321	35173
Florida	6788	9145	12045	15808	20054
Illinois	7320	9858	12882	16373	20969
Louisiana	6832	9136	11880	15195	18735

```
Massachusetts        6431        8851        12095       16360       21323
Michigan             6932        9777        13411       18061       23549
New Jersey           14977       21853       30678       41802       55188
New York             93055       130932      175808      228218      287866
Pennsylvania         4368        6163        8508        11353       14785
Washington           17925       21132       24609       28639       33104

[10 rows x 67 columns]
```

[23]:
```
import seaborn as sns
plt.figure(figsize=(12, 6))
for state in cum_sum_state.index:
    plt.plot(date_columns, cum_sum_state.loc[state], label=state)

plt.xlabel("Date")
plt.ylabel("Cumulative Cases")
plt.title("Cumulative COVID-19 Cases by State")
plt.xticks(rotation=45)  # Rotate x-axis labels for readability
plt.legend(title="State", bbox_to_anchor=(1.05, 1), loc='upper left')  # Place legend outside plot
# sns.despine()
```

[23]:
```
<matplotlib.legend.Legend at 0x174edbfb0>
```



2.6 3. Patient Data

`patients.csv` contains a small sample of patients from the world, which has following columns.

- **id:** the ID of the patient in this dataset.
- **reporting date:** the date of reporting.

- **location:** the state or the province where the patient is in.
- **country:** the country where the patient is in.
- **gender:** the gender of the patient (male = 0, female = 1) if the data is reported
- **age:** the age of the patient if the data is reported

1. Load the data from `patients.csv`

[24]:
```
pat_df = pd.read_csv(DATA_DIR + 'patients.csv')
pat_df.head()
```

[24]:

	id	reporting date	location	country	gender	age
0	1	1/20/2020	Shenzhen, Guangdong	China	0.0	66.0
1	2	1/20/2020	Shanghai	China	1.0	56.0
2	3	1/21/2020	Zhejiang	China	0.0	46.0
3	4	1/21/2020	Tianjin	China	1.0	60.0
4	5	1/21/2020	Tianjin	China	0.0	58.0

2. In the data set, the gender is coded as 0s and 1s (male=0, female=1). Please recode them as male and female in a new column. Also, please be aware that there are missing data in the dataset.

[25]:
```
pat_df['new_gender'] = pat_df['gender'].map({0: 'Male', 1: 'Female'})
pat_df
```

[25]:

	id	reporting date	location	country	gender	age	new_gender
0	1	1/20/2020	Shenzhen, Guangdong	China	0.0	66.0	Male
1	2	1/20/2020	Shanghai	China	1.0	56.0	Female
2	3	1/21/2020	Zhejiang	China	0.0	46.0	Male
3	4	1/21/2020	Tianjin	China	1.0	60.0	Female
4	5	1/21/2020	Tianjin	China	0.0	58.0	Male
…	…	…	…	…	…	…	…
1080	1081	2/25/2020	Innsbruck	Austria	NaN	24.0	NaN
1081	1082	2/24/2020	Afghanistan	Afghanistan	NaN	35.0	NaN
1082	1083	2/26/2020	Algeria	Algeria	0.0	NaN	Male
1083	1084	2/25/2020	Croatia	Croatia	0.0	NaN	Male
1084	1085	2/25/2020	Bern	Switzerland	0.0	70.0	Male

```
[1085 rows x 7 columns]
```

3. What suggestions can you make for handling the missing gender data prior to performing any statistical analysis?

[26]:
```
print(pat_df['new_gender'].isna().sum())
#since we do age analysis later, I don't want to just drop the NaN rows. I will␣
 ↪impute them with the percentage found using value_counts.
num_males = 105 # i get this from 0.576 * 183
num_females = 78 # i get this from 0.424 * 183
impute_values = ['Male'] * num_males + ['Female'] * num_females
np.random.shuffle(impute_values)
pat_df.loc[pat_df['new_gender'].isna(), 'new_gender'] = impute_values
print(pat_df['new_gender'].isna().sum())
```

```
183
0
```

Around 17% of the gender column is NaN. Dropping them would still leave us with a considerable portion of our dataset. Or we could randomly fill half with male and half with female to keep a similar gender ratio. This adds noise to our data. Or we could randomly fill it with the same proportion of male to female in the dataset. For example, if currently there are 60% females then 60% of the missing data would also be filled with females. This at least keeps the same gender ratio we have for the non NaN data.

In the following parts, we will ask you to do some basic statistical analysis. For the tasks below, the main statistic model we will use is goodness of fit, which will let you know how well the data we observed (e.g. the patient data here) fit to the distribution of the general population.

For example, if we assume that we are in a world where people mutually exclusively like sparkling water, tea, or coffee. We also know that in that world, 60% of people from country A, 30% from country B, and 10% from country C. Then naturally, if the country of origin doesn't affect which type of drink people like, when having a representative sample of the population, we should expect that for all types of drinks, the composition of people who like the drink should be the same as the general population (i.e. 6:3:1 in terms of the origin country), which is what we called a null hypothesis. However, if we observe that the compostion is *very* different from the composition of general population, then it indicates that there might be a correlation between where people are from and what types of drink they like. The more the observation data deviates from the expectation, the more likely that there is a correlation between them.

We often use p-value to evaluate whether the deviance is big enough to be considered as a correlation instead of a coincidence. In most cases, we would say that if p-value < 0.05, then the correlation is statistically significant. To calculate a p-value, one needs to decide which statistical test to use, which is related to the type of data one is dealing with. For example, in the above example, both dependent variable (i.e. types of drink) and independent variable (i.e. the origin country of the person) are categorical, in which case a Chi-square test should be used (read the links for more information).

4. What is the percentage of each gender in the dataset? We want to understand if there is a significant difference in gender among the covid patient data available to us. To conduct such a test, please write down your null/alternative hypothesis. Then provide a justification of which significant test should be used, the assumptions you made for using the test, and provide a p-value.

[27]:
```
print(pat_df['new_gender'].value_counts()/len(pat_df))
print(pat_df['new_gender'].value_counts())
```

```
new_gender
Male      0.576037
Female    0.423963
Name: count, dtype: float64
new_gender
Male      625
Female    460
Name: count, dtype: int64
```

Within the existing non null data, 57.6% are male, 42.4% are female. To test whether there is a significant difference in gender among the covid patient data our null hypothesis should be proportion of each gender = 50% (p_m = 0.5). That would mean the expected number of males and females are 542.5 each. The alternate hypothesis is p_m is not 0.5, or p_m > 0.5. I would choose p_m > 0.5 because it seems from our available data that there are more males than females. Thus it would be a one tailed z-test because we probably know the standard deviation of distribution of gender within the population. We will have an alpha significance value of a = 0.05.

[28]:
```
import scipy.stats as stats
n = len(pat_df)
x_males = 625
p_0 = 0.5

p_hat = x_males / n

# Z-test statistic calculation
z_score = (p_hat - p_0) / np.sqrt((p_0 * (1 - p_0)) / n)

# Calculate p-value (two-tailed test)
p_value = 1 - stats.norm.cdf(z_score)

# Output results
print(f"Z-score: {z_score}")
print(f"P-value: {p_value}")
```

```
Z-score: 5.009208110931056
P-value: 2.7327224405571116e-07
```

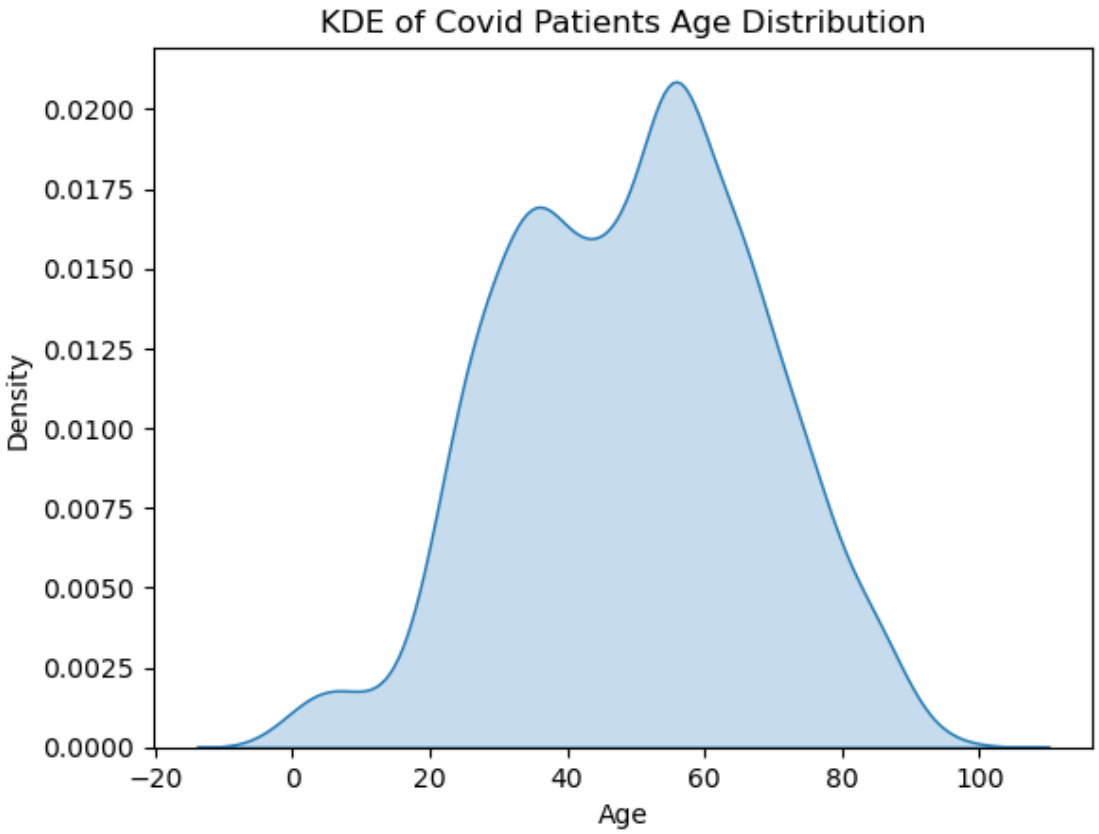
It's significant! This means that we reject the null and conclude that the proportion of males is greater for covid cases than in the population.

5. For numerical data such as age, provide a visualization that illustrates a good understanding of its distribution in the patient data with an explanation for your choice of plot.

[29]:
```
sns.kdeplot(x=pat_df['age'], shade=True)
plt.title('KDE of Covid Patients Age Distribution')
plt.xlabel('Age')
plt.ylabel('Density')
```

```
/var/folders/xm/ynhl86150vlb4bv7dtln_t0h0000gn/T/ipykernel_8495/400261796.py:1:
FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(x=pat_df['age'], shade=True)
```

[29]:
```
Text(0, 0.5, 'Density')
```



I chose a KDE because it is simple/easy to understand and appropriate for continuous numeric data. Percentage gives us a better understanding of the probability distribution than frequency does.

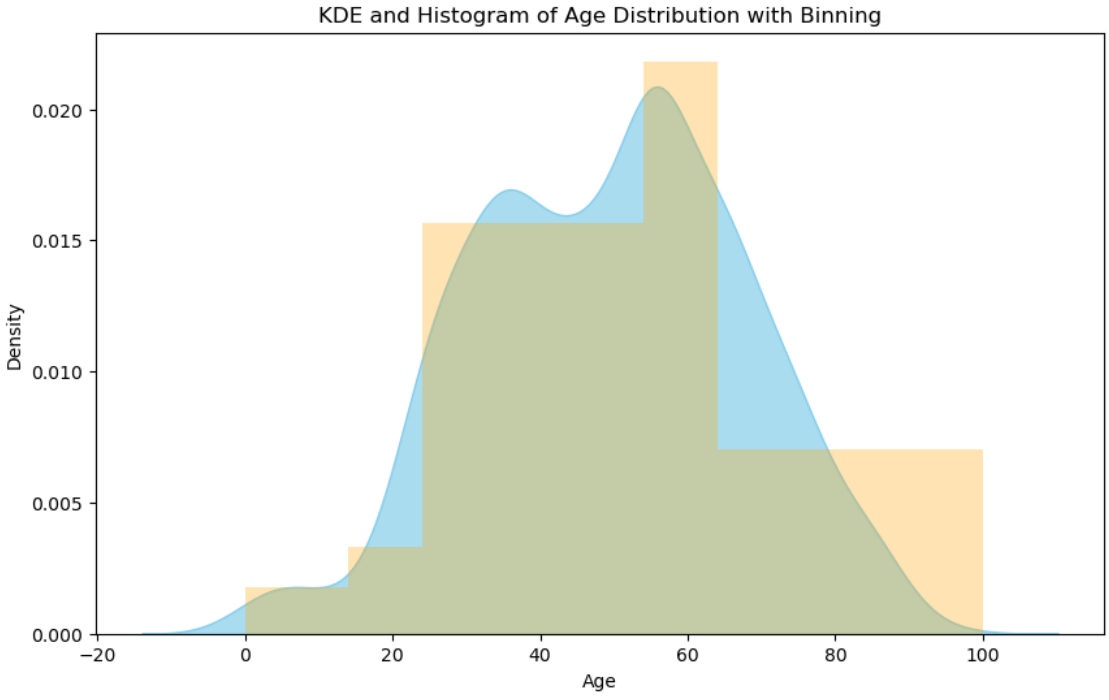
The world age structure data in 2018 is shown as below. * 0-14 years: 25.29% * 15-24 years: 15.77% * 25-54 years: 41.03% * 55-64 years: 8.84% * 65 years and over: 9.06%

Use this insight for the rest of the exercise.

6. Please plot the age distribution by discretizing the available data into bins used in the provided age structure.

[30]:
```
bins = [0, 14, 24, 54, 64, 100]

# Plotting KDE with custom bandwidth
plt.figure(figsize=(10, 6))

# KDE plot with bandwidth adjustment
sns.kdeplot(pat_df['age'], fill=True, color="skyblue", alpha=0.7)

# Overlay the histogram with the same bins for reference
plt.hist(pat_df['age'], bins=bins, alpha=0.3, color="orange", density=True)

# Set the title and labels
plt.title('KDE and Histogram of Age Distribution with Binning')
plt.xlabel('Age')
plt.ylabel('Density')
```

[30]:
```
Text(0, 0.5, 'Density')
```



7. Conduct a test to see whether the age distribution of patients in this dataset significantly deviates from the age structure of general population. Provide the null/alternative hypothesis, the test you are using, the assumptions of the data distribution, and the p-value.
   Also specify how you have handled the missing data for this specific question.

The null hypothesis is proportion of the age groups in the data set are the same proportion as the population. The alternate hypothesis is that they are different. Thus, this is a chi-squared test for goodness of fit. Using the chi-squared test means that each group must have at least 5 observations (they do). The p-value for our test is 0.05. Missing data will be imputed with the median age.

[31]:
```
#dealing with missing data
print(pat_df['age'].isna().sum())
pat_df['age'] = pat_df['age'].fillna(pat_df['age'].median())
```

```
242
```

[32]:
```
#getting counts
labels = ["0-14", "15-24", "25-54", "55-64", "65+"]
pat_df["age_group_bins"] = pd.cut(pat_df["age"], bins=bins, labels=labels,␣
 ↪right=True)
observed_counts = pat_df["age_group_bins"].value_counts().sort_index().values
# population_proportions = np.array([0.2529, 0.1577, 0.4103, 0.0884, 0.0906])
# expected_counts = 1085 * population_proportions

# ran the above code, it didn't add up to 1085 exactly due to rounding so I've␣
 ↪taken the liberty to do that here
expected_counts = np.array([274, 171, 445,  96 ,  99])
```

[33]:
```
import scipy.stats as stats

chi2_stat, p_value = stats.chisquare(observed_counts, expected_counts)

# Print results
print(f"Chi-Squared Statistic: {chi2_stat}")
print(f"p-value: {p_value}")
```

```
Chi-Squared Statistic: 613.6795890755858
p-value: 1.6962707152495943e-131
```

Our chi-squared test was significant because the p-value is very close to 0. Thus, we can reject the null hypothesis and conclude that the distribution of age within covid cases is different from distribution of age in population.